Active Object

Active Object · Serialize ledger mutations on one servant · Build with Waqar

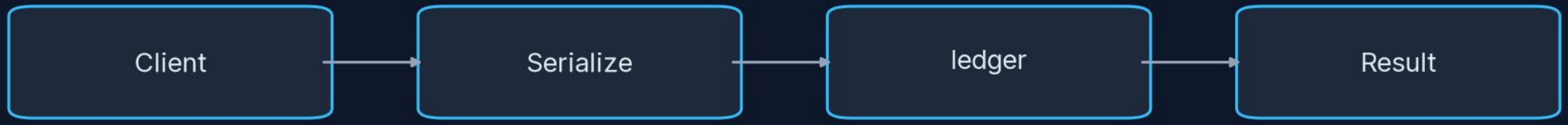
Today's Hook

- Problem: Serialize ledger mutations on one servant
- Pattern: Active Object — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Serialize ledger mutations on one servant
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Active Object: Serialize ledger mutations on one servant

Real-World Scenario

- Running example: Serialize ledger mutations on one servant
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.activeobject.problem;

import java.util.*;

public final class NaiveLedger {
  private int bal; public void deposit(int c){ bal+=c; } public int bal(){ return bal; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.activeobject.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class ActiveLedger implements AutoCloseable {
  private int bal; private final ExecutorService servant=Executors.newSingleThreadExecutor();
  CompletableFuture<Void> deposit(int c){ return CompletableFuture.runAsync(() -> { bal+=c; }, servant); }
  CompletableFuture<Integer> balance(){ return CompletableFuture.supplyAsync(() -> bal, servant); }
  public void close(){ servant.shutdown(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.activeobject.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Active Object: serialize ledger ===");
    System.out.println("--- naive ---"); var n=new NaiveLedger(); n.deposit(5); System.out.println(n.bal());
System.out.println("--- active object ---");
try (var a=new ActiveLedger()) {
  a.deposit(10).join(); a.deposit(5).join();
  System.out.println("bal="+a.balance().join());
}
  }
}
```

Walkthrough Output

```
=== Active Object: serialize ledger ===
--- naive ---
5
--- active object ---
bal=15
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Active Object.

Related Patterns

- Contrast Active Object with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Active Object → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.